

# HTML & CSS

Complete Beginner to Advanced Guide

Structure • Styling • Layout • Flexbox • Grid • Responsiveness

20 Pages

Hands-On Examples

Code Snippets

Best Practices

# Table of Contents

|            |                                       |         |
|------------|---------------------------------------|---------|
| Chapter 1  | Introduction to HTML                  | Page 3  |
| Chapter 2  | HTML Document Structure               | Page 4  |
| Chapter 3  | HTML Text & Lists                     | Page 5  |
| Chapter 4  | HTML Links, Images & Tables           | Page 6  |
| Chapter 5  | HTML Forms & Input Elements           | Page 7  |
| Chapter 6  | HTML5 Semantic Elements               | Page 8  |
| Chapter 7  | Introduction to CSS                   | Page 9  |
| Chapter 8  | CSS Selectors & Specificity           | Page 10 |
| Chapter 9  | CSS Box Model                         | Page 11 |
| Chapter 10 | CSS Typography & Colors               | Page 12 |
| Chapter 11 | CSS Display & Positioning             | Page 13 |
| Chapter 12 | CSS Flexbox Layout                    | Page 14 |
| Chapter 13 | CSS Grid Layout                       | Page 15 |
| Chapter 14 | CSS Transitions & Animations          | Page 16 |
| Chapter 15 | Responsive Web Design & Media Queries | Page 17 |
| Chapter 16 | CSS Variables & Custom Properties     | Page 18 |
| Chapter 17 | CSS Pseudo-classes & Pseudo-elements  | Page 19 |
| Chapter 18 | Best Practices & Summary              | Page 20 |

## Chapter 1

# Introduction to HTML

HTML (HyperText Markup Language) is the standard language used to create and structure content on the web. It uses a system of elements represented by tags to define headings, paragraphs, links, images, and more. Every web page you visit is built on HTML.

## What is HTML?

HTML is not a programming language — it is a markup language. It describes the structure of a webpage using a series of tags. Each tag tells the browser how to display the content enclosed within it. HTML documents are plain text files saved with a **.html** extension.

A simple HTML tag looks like this:

```
<tagname> content goes here </tagname>
```

## Key HTML Concepts

- **Tags:** Keywords surrounded by angle brackets, e.g., `<p>`, `<h1>`, `<div>`.
- **Elements:** An opening tag + content + closing tag forms an element.
- **Attributes:** Extra information added inside the opening tag, e.g., `id`, `class`, `href`.
- **Nesting:** Elements placed inside other elements to create hierarchy.
- **Comments:** Non-rendered notes using `<!-- comment -->` syntax.

### Pro Tip

Always validate your HTML using the W3C Markup Validation Service at [validator.w3.org](https://validator.w3.org) to catch syntax errors early.

## Chapter 2

# HTML Document Structure

Every HTML document follows a standard structure. This skeleton provides the browser with essential metadata and tells it where the visible page content begins.

### The Basic Skeleton

```
<!DOCTYPE html>
<html lang='en'>
<head>
<meta charset='UTF-8'>
<meta name='viewport' content='width=device-width, initial-scale=1.0'>
<title>My First Page</title>
<link rel='stylesheet' href='style.css'>
</head>
<body>
<h1>Hello, World!</h1>
<p>Welcome to my website.</p>
</body>
</html>
```

### Breaking Down Each Part

| Element         | Purpose                                                |
|-----------------|--------------------------------------------------------|
| <!DOCTYPE html> | Declares the document type as HTML5                    |
| <html>          | Root element wrapping all content; lang sets language  |
| <head>          | Contains metadata (not displayed on screen)            |
| <meta charset>  | Sets character encoding (UTF-8 supports all languages) |
| <meta viewport> | Controls layout on mobile devices                      |
| <title>         | Text shown in the browser tab                          |
| <link>          | Links external stylesheets or resources                |
| <body>          | Contains all visible page content                      |

## Chapter 3

# HTML Text & Lists

---

## Headings

HTML provides six levels of headings (<h1> through <h6>). <h1> is the most important (largest) and should be used once per page for the main title. Headings help with both readability and SEO.

```
<h1>Main Title</h1>
<h2>Section Heading</h2>
<h3>Sub-section</h3>
<h4>Smaller heading</h4>
```

## Paragraphs & Text Formatting

```
<p>This is a paragraph of text.</p>
<b>Bold text</b> or <strong>Strong text</strong>
<i>Italic text</i> or <em>Emphasized text</em>
<u>Underlined</u> <mark>Highlighted</mark>
<br> <!-- Line break -->
<hr> <!-- Horizontal rule -->
```

## HTML Lists

Unordered list (bullet points):

```
<ul>
<li>Item One</li>
<li>Item Two</li>
</ul>
```

Ordered list (numbered):

```
<ol>
<li>First step</li>
<li>Second step</li>
</ol>
```

Description list:

```
<dl>
<dt>HTML</dt>
<dd>HyperText Markup Language</dd>
</dl>
```

## Chapter 4

# HTML Links, Images & Tables

---

## Hyperlinks

```
<!-- Basic link -->
<a href='https://www.example.com'>Visit Example</a>

<!-- Open in new tab -->
<a href='page.html' target='_blank'>New Tab Link</a>

<!-- Email link -->
<a href='mailto:info@example.com'>Email Us</a>
```

## Images

```
<img src='photo.jpg' alt='A beautiful landscape' width='400' height='300'>

<!-- Always use alt attribute for accessibility -->
<img src='logo.png' alt='Company Logo' class='logo'>
```

## HTML Tables

Tables are used to display structured, tabular data. They should NOT be used for layout.

```
<table>
<thead>
<tr>
<th>Name</th> <th>Age</th> <th>City</th>
</tr>
</thead>
<tbody>
<tr>
<td>Alice</td> <td>30</td> <td>Mumbai</td>
</tr>
</tbody>
</table>
```

### Accessibility

Always add the alt attribute to images and use <th> for table headers. Screen readers rely on these to describe content to visually impaired users.

## Chapter 5

# HTML Forms & Input Elements

Forms allow users to input data that is sent to a server for processing. They are essential for login pages, search bars, registrations, and more.

```
<form action='/submit' method='POST'>

<label for='name'>Full Name:</label>
<input type='text' id='name' name='name' placeholder='Enter name'>

<label for='email'>Email:</label>
<input type='email' id='email' name='email' required>

<label for='pass'>Password:</label>
<input type='password' id='pass' name='password'>

<label>Gender:</label>
<input type='radio' name='gender' value='male'> Male
<input type='radio' name='gender' value='female'> Female

<label>Agree to Terms:</label>
<input type='checkbox' name='agree'>

<select name='country'>
<option value='in'>India</option>
<option value='us'>USA</option>
</select>

<textarea name='message' rows='4'></textarea>

<button type='submit'>Submit</button>
</form>
```

### Common input types:

- **type='text'** — Single-line text input
- **type='email'** — Validates email format
- **type='password'** — Masked input field
- **type='number'** — Numeric input with arrows
- **type='date'** — Calendar date picker
- **type='file'** — File upload button
- **type='submit'** — Submits the form
- **type='reset'** — Resets all fields

## Chapter 6

# HTML5 Semantic Elements

Semantic elements clearly describe their meaning to both the browser and the developer. Using semantic HTML improves accessibility, SEO, and code readability.

```
<header>Site header / navigation brand</header>
<nav>Navigation links</nav>
<main>
<article>
<h2>Blog Post Title</h2>
<section>Intro section</section>
<section>Body section</section>
</article>
<aside>Sidebar / related links</aside>
</main>
<footer>Copyright, links</footer>
```

### Semantic vs Non-Semantic

| Non-Semantic (Avoid)  | Semantic (Preferred) |
|-----------------------|----------------------|
| <div id='header'>     | <header>             |
| <div id='nav'>        | <nav>                |
| <div id='main'>       | <main>               |
| <div id='footer'>     | <footer>             |
| <div class='article'> | <article>            |

## Chapter 7

# Introduction to CSS

CSS (Cascading Style Sheets) is the language used to style HTML documents. It controls colors, fonts, spacing, layout, and much more. CSS separates presentation from structure, making websites easier to maintain.

## Three Ways to Add CSS

1. **Inline CSS** — applied directly to an element (least preferred):

```
<p style='color: red; font-size: 16px;'>Red text</p>
```

2. **Internal CSS** — inside a `<style>` tag in the `<head>`:

```
<style>
p { color: blue; font-size: 16px; }
</style>
```

3. **External CSS** — linked stylesheet file (best practice):

```
<link rel='stylesheet' href='style.css'>

/* style.css */
p {
color: navy;
font-size: 16px;
line-height: 1.6;
}
```

## CSS Rule Anatomy

```
selector {
property: value; /* declaration */
font-size: 18px;
color: #333;
}
```

A CSS rule consists of a **selector** that targets HTML elements, and one or more **declarations** (property-value pairs) inside curly braces.

# Chapter 8

## CSS Selectors & Specificity

```
/* Type selector */
h1 { color: blue; }

/* Class selector */
.card { border: 1px solid #ccc; }

/* ID selector */
#header { background: #000; }

/* Universal selector */
* { box-sizing: border-box; }

/* Descendant selector */
nav a { text-decoration: none; }

/* Child selector */
ul > li { list-style: square; }

/* Adjacent sibling */
h2 + p { margin-top: 0; }

/* Attribute selector */
input[type='email'] { border-color: green; }
```

### Specificity Rules

When multiple rules target the same element, CSS uses specificity to decide which wins. Higher specificity overrides lower.

| Selector Type                           | Specificity Points |
|-----------------------------------------|--------------------|
| Inline style                            | 1000               |
| ID selector (#id)                       | 100                |
| Class (.class), attribute, pseudo-class | 10                 |
| Type (element), pseudo-element          | 1                  |
| Universal (*)                           | 0                  |

## Chapter 9

# CSS Box Model

Every HTML element is a rectangular box. The CSS Box Model describes the layers that make up this box: content, padding, border, and margin.

```
.box {  
width: 300px; /* Content width */  
height: 150px; /* Content height */  
padding: 20px; /* Space inside border */  
border: 3px solid #333; /* Border line */  
margin: 30px; /* Space outside border */  
}  
  
/* Total width = width + padding*2 + border*2 */  
/* = 300 + 40 + 6 = 346px (default box-sizing) */  
  
/* Use border-box to include padding/border in width: */  
* { box-sizing: border-box; }
```

## Margin & Padding Shorthand

```
/* All four sides */  
margin: 10px;  
  
/* Top/bottom Left/right */  
padding: 10px 20px;  
  
/* Top Right Bottom Left (clockwise) */  
margin: 10px 20px 15px 5px;  
  
/* Individual sides */  
padding-top: 10px;  
padding-right: 15px;  
padding-bottom: 10px;  
padding-left: 15px;
```

### Best Practice

Always set `* { box-sizing: border-box; }` at the top of your stylesheet. It makes width calculations much more intuitive and predictable.

## Chapter 10

# CSS Typography & Colors

---

```
body {
font-family: 'Segoe UI', Arial, sans-serif;
font-size: 16px;
font-weight: 400; /* 100-900 or bold/normal */
font-style: italic;
line-height: 1.6; /* Recommended for readability */
letter-spacing: 0.5px;
text-align: justify; /* left | center | right | justify */
text-transform: uppercase; /* none | capitalize | lowercase */
text-decoration: underline; /* none | underline | line-through */
}
```

## CSS Colors

```
/* Named colors */
color: red; background-color: white;

/* Hex (most common) */
color: #2563EB; /* 6-digit hex */
color: #fff; /* 3-digit shorthand */

/* RGB */
color: rgb(37, 99, 235);

/* RGBA (with opacity 0-1) */
background: rgba(0, 0, 0, 0.5); /* 50% transparent black */

/* HSL (Hue, Saturation%, Lightness%) */
color: hsl(221, 83%, 53%);

/* Gradients */
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

## Chapter 11

# CSS Display & Positioning

---

## The display Property

```
display: block; /* Full width, starts on new line */
display: inline; /* Sits in line with text, no width/height */
display: inline-block; /* Inline but accepts width/height */
display: none; /* Hides element completely */
display: flex; /* Enables Flexbox */
display: grid; /* Enables CSS Grid */
```

## CSS Position

```
/* Static (default) - normal flow */
position: static;

/* Relative - offset from normal position */
position: relative;
top: 10px; left: 20px;

/* Absolute - removed from flow, positioned to nearest
positioned ancestor */
position: absolute;
top: 0; right: 0;

/* Fixed - stays fixed on screen when scrolling */
position: fixed;
bottom: 20px; right: 20px;

/* Sticky - relative until threshold, then fixed */
position: sticky;
top: 0;
```

## Z-index

The **z-index** property controls stacking order of positioned elements. Higher z-index = in front. Only works with non-static positioning.

```
.modal { position: fixed; z-index: 1000; }
.overlay { position: fixed; z-index: 999; }
```

## Chapter 12

# CSS Flexbox Layout

Flexbox (Flexible Box Layout) is a one-dimensional layout system perfect for arranging items in a row or column with powerful alignment control.

```
.container {  
  display: flex;  
  flex-direction: row; /* row | column | row-reverse */  
  flex-wrap: wrap; /* nowrap | wrap | wrap-reverse */  
  justify-content: center; /* Main axis alignment */  
  align-items: center; /* Cross axis alignment */  
  gap: 16px; /* Space between items */  
}  
  
.item {  
  flex: 1; /* Grow equally */  
  flex-grow: 1;  
  flex-shrink: 0;  
  flex-basis: 200px;  
  align-self: flex-start; /* Override align-items for one item */  
  order: 2; /* Change visual order */  
}
```

### justify-content values:

- **flex-start** — Pack items to start (default)
- **flex-end** — Pack items to end
- **center** — Center items
- **space-between** — Equal space between items, none at edges
- **space-around** — Equal space around each item
- **space-evenly** — Perfectly equal spacing including edges

🔗 **Common Use** To center an element both horizontally and vertically: `display: flex; justify-content: center; align-items: center;` on the parent. This is the easiest centering method in CSS.

Chapter 13

# CSS Grid Layout

CSS Grid is a two-dimensional layout system that allows you to create complex row-and-column layouts with precision. It's the most powerful layout tool in CSS.

```
.grid-container {
display: grid;
grid-template-columns: 1fr 2fr 1fr; /* 3 cols, middle twice as wide */
grid-template-rows: auto 1fr auto; /* header, main, footer */
gap: 20px; /* row-gap column-gap */
}

/* Repeat notation */
grid-template-columns: repeat(4, 1fr); /* 4 equal columns */

/* Auto-fill responsive grid */
grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));

/* Place items */
.header { grid-column: 1 / -1; } /* Span all columns */
.sidebar { grid-row: 2 / 4; } /* Span rows 2-3 */
.item { grid-area: 2 / 1 / 3 / 3; } /* row-start/col-start/end/end */
```

## Grid vs Flexbox

| Feature         | Flexbox                | Grid                  |
|-----------------|------------------------|-----------------------|
| Dimensions      | 1D (row OR column)     | 2D (rows AND columns) |
| Best for        | Navigation, components | Page layouts          |
| Content-driven? | Yes                    | No (layout-first)     |
| Browser support | Excellent              | Excellent             |

## Chapter 14

# CSS Transitions & Animations

---

## CSS Transitions

Transitions let you animate changes between two states smoothly.

```
.button {
background: blue;
transition: background 0.3s ease, transform 0.2s;
}

.button:hover {
background: darkblue;
transform: scale(1.05);
}

/* Transition properties */
transition-property: all;
transition-duration: 0.4s;
transition-timing-function: ease-in-out; /* or linear, ease, cubic-bezier */
transition-delay: 0.1s;
```

## CSS Animations

```
@keyframes slideIn {
from {
opacity: 0;
transform: translateX(-100px);
}
to {
opacity: 1;
transform: translateX(0);
}
}

.element {
animation: slideIn 0.5s ease forwards;
/* name duration timing fill-mode */
}

/* Infinite loop */
.spinner {
animation: spin 1s linear infinite;
}

@keyframes spin {
from { transform: rotate(0deg); }
to { transform: rotate(360deg); }
}
```

## Chapter 15

# Responsive Web Design & Media Queries

Responsive design ensures your website looks good on all screen sizes — from mobile phones to large desktop monitors.

## The Viewport Meta Tag

```
<meta name='viewport' content='width=device-width, initial-scale=1.0'>
```

## Media Queries

```
/* Mobile first approach (recommended) */

/* Base styles: mobile */
.container { padding: 16px; }
.grid { grid-template-columns: 1fr; }

/* Tablet (768px and up) */
@media (min-width: 768px) {
  .container { padding: 24px; }
  .grid { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop (1024px and up) */
@media (min-width: 1024px) {
  .container { max-width: 1200px; margin: 0 auto; }
  .grid { grid-template-columns: repeat(3, 1fr); }
}

/* Print media query */
@media print {
  nav, footer { display: none; }
}
```

## Common Breakpoints

- **320px – 480px** — Small mobile phones
- **481px – 768px** — Tablets & large phones
- **769px – 1024px** — Laptops & small desktops
- **1025px+** — Large desktops & TVs

## Chapter 16

# CSS Variables & Custom Properties

CSS Custom Properties (also known as CSS Variables) allow you to store reusable values and create themes with ease. They're defined with `--` and accessed with `var()`.

```
/* Define variables on :root (global scope) */
:root {
  --primary: #2563EB;
  --secondary: #EA580C;
  --text: #1E293B;
  --bg: #F8FAFC;
  --radius: 8px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 32px;
  --font-base: 'Inter', sans-serif;
}

/* Use variables */
body {
  color: var(--text);
  background: var(--bg);
  font-family: var(--font-base);
}

.button {
  background: var(--primary);
  border-radius: var(--radius);
  padding: var(--spacing-sm) var(--spacing-md);
}

/* Override in a specific scope */
.dark-mode {
  --text: #F1F5F9;
  --bg: #0F172A;
}
```

### 🔦 Dark Mode

Using CSS Variables makes implementing dark mode simple. Just redefine the color variables on a `.dark` class or `prefers-color-scheme` media query and all elements update automatically.

## Chapter 17

# CSS Pseudo-classes & Pseudo-elements

---

## Pseudo-classes (single colon :)

Pseudo-classes select elements based on their state or position.

```
a:hover { color: red; } /* Mouse over */
a:visited { color: purple; } /* Already visited */
a:active { color: orange; } /* Being clicked */
input:focus { outline: 2px solid blue; } /* Focused */

/* Structural pseudo-classes */
li:first-child { font-weight: bold; }
li:last-child { border-bottom: none; }
li:nth-child(2) { color: blue; }
li:nth-child(odd) { background: #f0f0f0; }
li:nth-child(even) { background: #ffffff; }

p:not(.special) { color: gray; } /* Negation */
input:required { border-color: red; }
input:valid { border-color: green; }
```

## Pseudo-elements (double colon ::)

Pseudo-elements create virtual elements or target specific parts of an element.

```
p::first-line { font-weight: bold; }
p::first-letter { font-size: 2em; float: left; }

/* ::before and ::after – inject content */
.card::before {
  content: '';
  display: block;
  width: 4px;
  height: 100%;
  background: var(--primary);
  position: absolute;
  left: 0; top: 0;
}

/* Selection highlight */
::selection { background: #2563EB; color: white; }
```

Chapter 18

Best Practices & Summary

HTML Best Practices

- ✓ Always declare `<!DOCTYPE html>` at the top of every HTML file.
- ✓ Use semantic elements (`<header>`, `<main>`, `<article>`) over generic `<div>` tags.
- ✓ Add alt text to all images for accessibility and SEO.
- ✓ Use `<label>` elements with every form input.
- ✓ Keep HTML structure clean and well-indented for readability.
- ✓ Validate your HTML at [validator.w3.org](https://validator.w3.org) to catch errors.

CSS Best Practices

- ✓ Set `* { box-sizing: border-box; }` globally at the start.
- ✓ Use CSS custom properties (variables) for colors, spacing, and fonts.
- ✓ Follow mobile-first design with min-width media queries.
- ✓ Prefer Flexbox and Grid over floats and positioning for layout.
- ✓ Organize CSS with a consistent naming convention like BEM.
- ✓ Minimize use of `!important` — it breaks the cascade.
- ✓ Use shorthand properties (margin, padding, font) to keep code concise.

What to Learn Next

| JavaScript           | Add interactivity, DOM manipulation, events      |
|----------------------|--------------------------------------------------|
| CSS Frameworks       | Bootstrap, Tailwind CSS for rapid development    |
| Sass/SCSS            | CSS preprocessor with variables, nesting, mixins |
| Accessibility (a11y) | ARIA roles, keyboard navigation, contrast ratios |
| Performance          | Lazy loading, minification, critical CSS         |
| Version Control      | Git & GitHub for tracking code changes           |

Congratulations! You've completed the HTML & CSS Complete Guide. Keep practising by building real projects — a personal portfolio, a blog, or a product landing page are great starting points. Happy coding!